第1课：神经网络基础

- 李伟昌, weichangli@zju.edu.cn
- 浙江大学**海洋人工智能**课程讲义

- 版本 1, 2026年5月20日

时长： 45分钟
目标： 理解神经网络基础并实现一个基本网络

1.1 深度学习革命

历史时间线：从AI到深度学习

AI基础（1940年代前 - 1950年代）

```
1943 ──→ McCulloch-Pitts神经元：第一个神经元数学模型
         └→ 二元阈值单元，为神经计算奠定基础

1950 ──→ 图灵测试由艾伦·图灵提出
         └→ "机器能思考吗？" – AI的哲学基础

1956 ──→ 达特茅斯会议："人工智能"诞生
         └→ McCarthy、Minsky、Shannon、Rochester创造了这个术语
```

早期神经网络（1960年代 - 1980年代）

```
1958 ──→ 感知器（Rosenblatt）
         └→ 第一个学习算法，可以分类线性可分模式

1969 ──→ 感知器（Minsky & Papert）
         └→ 证明单层感知器无法解决XOR问题 → 第一次AI寒冬

1970年代 ──→ 专家系统时代
         └→ 手工编码规则（MYCIN、DENDRAL）占主导；神经网络被忽视

1982 ──→ Hopfield网络
         └→ 具有基于能量学习的循环网络

1986 ──→ 反向传播普及（Rumelhart、Hinton、Williams）
         └→ 实现多层网络训练；兴趣复苏
```

机器学习时代（1990年代 - 2000年代）

```
1990年代 ──→ 统计方法占主导
         ├→ 支持向量机（SVM）– Vapnik
         ├→ 随机森林 – Breiman
         └→ 核方法；神经网络处于"寒冬"

1997 ──→ LSTM（Hochreiter & Schmidhuber）
         └→ 解决了序列的梯度消失问题

1997 ──→ IBM深蓝击败卡斯帕罗夫
         └→ 专用硬件 + 搜索，而非学习

2006 ──→ 深度信念网络（Hinton）
         └→ "深度学习"术语流行；贪婪逐层预训练
```

深度学习革命（2010年代 - 至今）

```
2012 ──→ AlexNet赢得ImageNet（Krizhevsky、Sutskever、Hinton）
         └→ 15.3%错误率 vs 第二名26.2%；GPU训练；ReLU；Dropout

2014 ──→ GAN（Goodfellow等人）、VGGNet、ResNet（2015）
         └→ 生成模型；更深架构；跳跃连接

2016 ──→ AlphaGo击败李世石
         └→ 深度强化学习 + 蒙特卡洛树搜索

2017 ──→ Transformer架构（Vaswani等人）
         └→ "Attention Is All You Need" – 可并行、长距离依赖

2018 ──→ BERT（Devlin等人）
         └→ 双向预训练革新NLP

2019 ──→ GPT-2、GPT-3（2020）
         └→ 缩放定律出现；少样本学习能力

2022 ──→ ChatGPT、Stable Diffusion
         └→ 大规模公众采用；多模态生成

2023+ ──→ GPT-4、Claude、Gemini、Llama
         └→ 多模态大语言模型；推理能力；开源模型
```

通用近似定理

定理内容： 具有单个隐藏层且包含有限数量神经元的前馈网络，在激活函数满足温和假设的条件下，可以近似$\mathbb{R}^n$紧子集上的任何连续函数。

数学表达： 对于任何连续函数 $f : [0,1]^d \to \mathbb{R}$ 和任何 $\varepsilon > 0$，存在一个具有一个隐藏层的神经网络 N使得：

$$|f(x) - N(x)| < \varepsilon \forall x \in [0,1]^d$$

重要注意事项：

1. **存在性 ≠ 构造性：** 定理说明这样的网络存在；但没有告诉我们如何找到它
2. **宽度可能是指数级的：** 所需隐藏层宽度可能随输入维度指数增长
3. **深度更高效：** 深度网络可以用比浅层网络指数级更少的参数表示函数

为什么重要： 神经网络在理论上能够学习数据中的任何模式——问题在于找到正确的架构和训练程序，而非存在根本性限制。

```
直观可视化：

浅层网络（一个隐藏层）：      深度网络（多层）：
  输入 ──→ [██████] ──→ 输出      输入 ──→ [██]──→[██]──→[██]──→ 输出
             （很宽）                        （更窄，但更深）

两者都可以近似相同的函数，但深度网络对于
复杂的层次化模式通常更具参数效率。
```

为什么是现在？

因素	过去（1990年代）	现在（2020年代）
数据	MB级别	PB级别（互联网规模）
计算	MHz CPU	具有1000+核心的GPU/TPU
算法	浅层网络，糟糕的初始化	深层网络，批归一化，残差连接
工具	手工编写的C++	PyTorch、TensorFlow、JAX

关键应用

- 计算机视觉：目标检测、分割、生成
- 自然语言处理：翻译、摘要、问答
- 语音：识别、合成
- 机器人：控制、规划
- 科学：蛋白质折叠、药物发现、气候建模

1.2 神经元：构建模块

生物启发

生物神经元 → 人工神经元

- 树突（输入） → 加权求和
- 胞体（处理） → 激活函数
- 轴突（输出） → 输出信号

注意： 生物启发是松散的；我们优化数学可处理性。

数学定义

单个神经元计算：

$$y = f\left(\sum_{i=1}^{n} w_i x_i + b\right) = f(\mathbf{w}^T\mathbf{x} + b)$$

其中：

- $\mathbf{x} \in \mathbb{R}^n$ = 输入向量
- $\mathbf{w} \in \mathbb{R}^n$ = 权重向量
- $b \in \mathbb{R}$ = 偏置项
- f = 激活函数（非线性）

神经元可视化表示

```
  x₁ ──w₁──┐
           │
  x₂ ──w₂──┼──→ Σ ──→ f(·) ──→ y
           │    ↑
  x₃ ──w₃──┘    b
               （偏置）

  其中：Σ = w₁x₁ + w₂x₂ + w₃x₃ + b
```

计算图视角（受*Karpathy的micrograd*启发）：

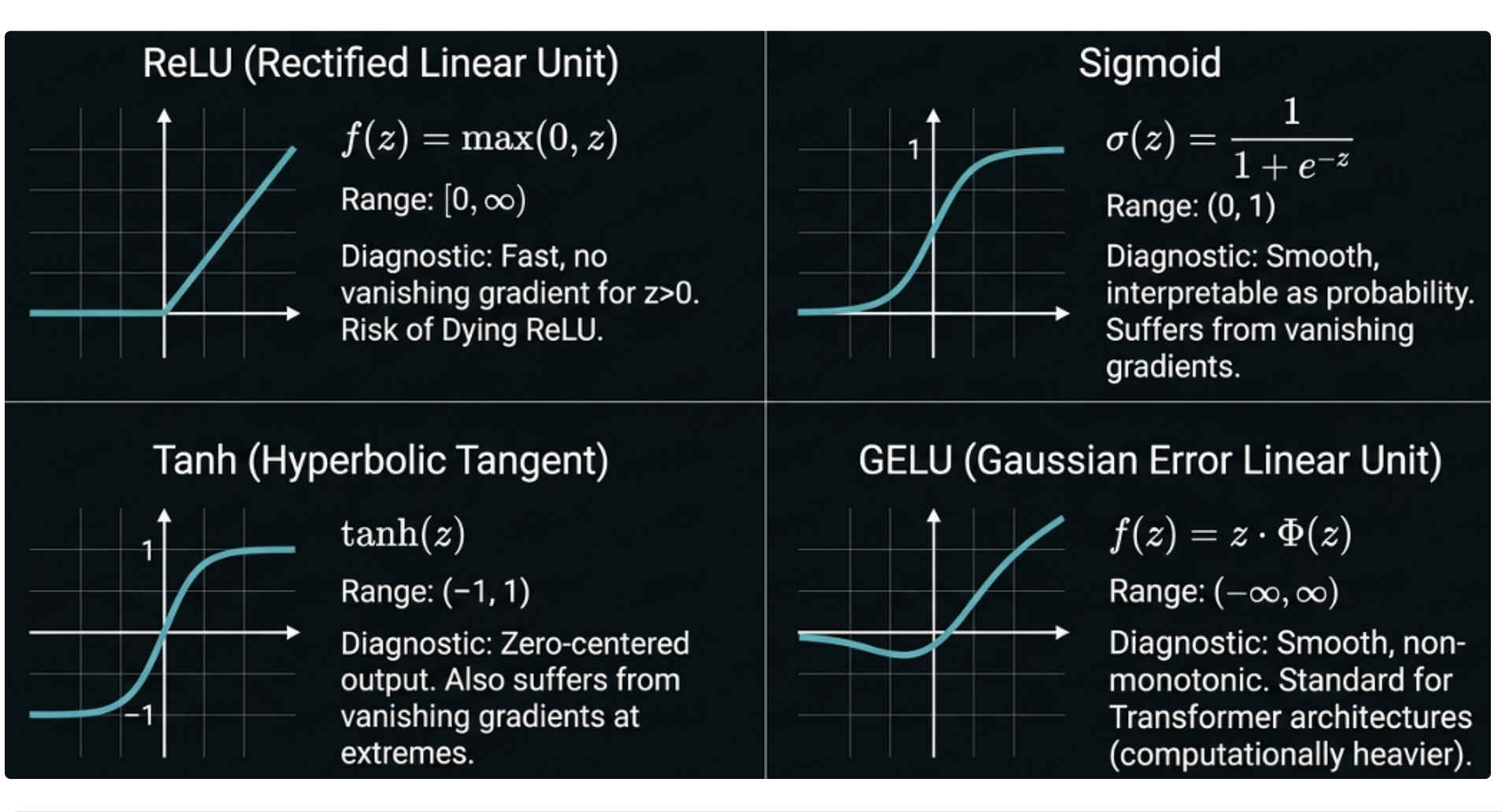
```
  x₁ ──[× w₁]──┐
               │
  x₂ ──[× w₂]──┼──[+]──[+]──[f]──→ y
               │   ↑    ↑
  x₃ ──[× w₃]──┘   b   sum
```

每个操作（乘法、加法、激活）都是计算图中的一个节点。这种图结构使得自动微分（反向传播）成为可能，我们将在第2课中看到。

激活函数

函数	公式	输出范围	优点	缺点
Sigmoid	$\sigma(z) = \frac{1}{1+e^{-z}}$	(0, 1)	平滑、可解释	梯度消失
Tanh	$\tanh(z)$	(-1, 1)	以零为中心	梯度消失
ReLU	$\max(0, z)$	[0, ∞)	无梯度消失、快速	"死亡ReLU"
Leaky ReLU	$\max(\alpha z, z)$	(-∞, ∞)	修复死亡ReLU	额外超参数
GELU	$z \cdot \Phi(z)$	(-∞, ∞)	平滑，用于Transformers	计算更昂贵

视觉对比：



```
为什么需要非线性？没有激活函数，堆叠层只是矩阵乘法：
  Layer2(Layer1(x)) = W₂(W₁x) = (W₂W₁)x = W'x  → 仍然只是线性变换！
```

推荐： 从ReLU开始；Transformers使用GELU。

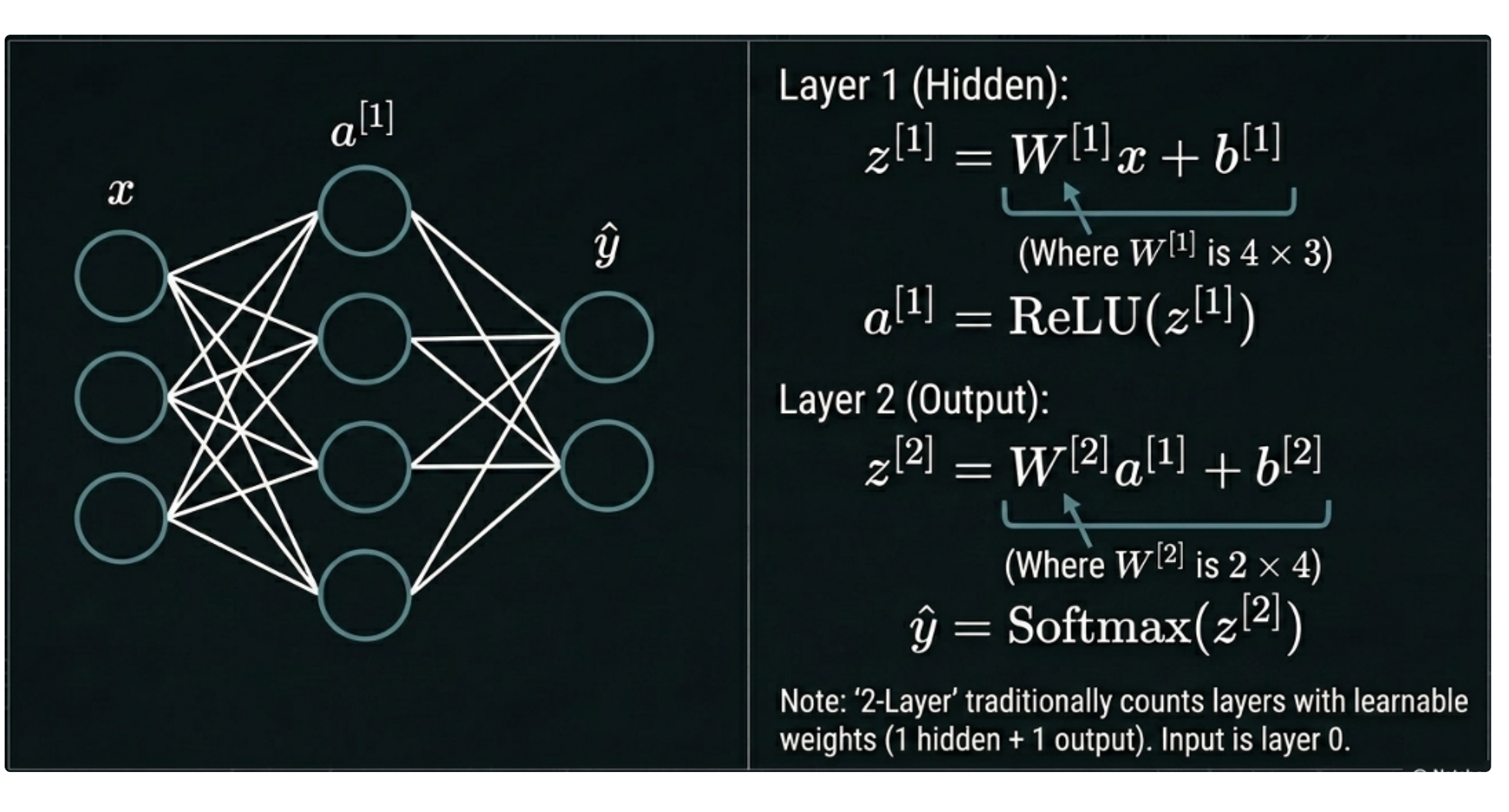
```
import torch
import torch.nn as nn

# PyTorch中的常见激活函数
relu = nn.ReLU()
gelu = nn.GELU()
sigmoid = nn.Sigmoid()
tanh = nn.Tanh()

# 示例
x = torch.randn(10)
output = relu(x)  # 所有负值变为0
```

1.3 神经网络架构

分层结构



x
$a^{[1]}$
$\hat{y}$
Layer 1 (Hidden):
$z^{[1]} = W^{[1]}x + b^{[1]}$
(Where $W^{[1]}$ is 4×3)
$a^{[1]} = \mathrm{ReLU}(z^{[1]})$
Layer 2 (Output):
$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$
(Where $W^{[2]}$ is 2×4)
$\hat{y} = \mathrm{Softmax}(z^{[2]})$
Note: '2-Layer' traditionally counts layers with learnable weights (1 hidden + 1 output). Input is layer 0.

全连接（密集）层

对于具有n_l个神经元的层l：

$$\mathbf{z}^{[l]} = \mathbf{W}^{[l]}\mathbf{a}^{[l-1]} + \mathbf{b}^{[l]}$$

$$\mathbf{a}^{[l]} = f(\mathbf{z}^{[l]})$$

其中$\mathbf{W}^{[l]} \in \mathbb{R}^{n_l \times n_{l-1}}$

网络深度和宽度

- **深度：** 隐藏层数量
- **宽度：** 每层神经元数量
- **通用近似定理：** 具有足够宽度的单个隐藏层可以近似任何连续函数（但深度网络更有效）

架构示例：2层网络

```
import torch.nn as nn

class TwoLayerNet(nn.Module):
    def __init__(self, input_size, hidden_size, num_classes):
        super().__init__()
        self.fc1 = nn.Linear(input_size, hidden_size)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(hidden_size, num_classes)

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu(x)
        x = self.fc2(x)
        return x
```

1.4 损失函数

回归损失

均方误差（MSE）：

$$\mathcal{L}_{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

平均绝对误差（MAE）：

$$\mathcal{L}_{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

特性： MSE对大错误惩罚更重；MAE对异常值更鲁棒。

分类损失

交叉熵损失（多分类）：

$$\mathcal{L}_{CE} = -\frac{1}{n}\sum_{i=1}^{n}\sum_{c=1}^{C} y_{i,c}\log(\hat{y}_{i,c})$$

其中$\hat{y}_{i,c}$是类别c的预测概率（来自softmax）。

二元交叉熵：

$$\mathcal{L}_{BCE} = -\frac{1}{n}\sum_{i=1}^{n}[y_i\log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)]$$

Softmax函数

将logits转换为概率：

$$\mathrm{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

特性：

- 输出总和为1
- 可微
- 放大差异（追求最大）

```
# PyTorch实现
import torch.nn.functional as F

# 多分类
output = model(x)
loss = F.cross_entropy(output, labels)  # 内部包含softmax

# 二分类
loss = F.binary_cross_entropy_with_logits(output, labels)
```

1.5 前向传播

前向传播可视化

网络架构流程：

```
输入层       隐藏层1       隐藏层2       输出层

  x₁ ──┐
       │
  x₂ ──┼──→ [Linear + ReLU] ──→ [Linear + ReLU] ──→ [Linear + Softmax] ──→ ŷ
       │          ↑                    ↑                     ↑
  x₃ ──┘    W¹, b¹ (5×3)        W², b² (5×5)          W³, b³ (2×5)

维度流程：
x (3×1) → z¹ (5×1) → a¹ (5×1) → z² (5×1) → a² (5×1) → z³ (2×1) → ŷ (2×1)
```

单个神经元的计算图（Karpathy的*micrograd*风格）：

```
前向传播：
============

   x₁ = 2.0 ──[× w₁=0.5]──┐
                           ├──[+]──→ z = 2.35 ──[tanh]──→ y = 0.98
   x₂ = 1.5 ──[× w₂=0.9]──┤               ↑
                           └──[+]── bias=0.1

每个节点：(数据, 梯度)  ← 我们存储值和梯度
```

算法步骤

1. 从输入$\mathbf{x}$开始
2. 对于每层$l = 1, 2, ..., L$：
    - 线性变换：$\mathbf{z}^{[l]} = \mathbf{W}^{[l]}\mathbf{a}^{[l-1]} + \mathbf{b}^{[l]}$
    - 激活：$\mathbf{a}^{[l]} = f(\mathbf{z}^{[l]})$
3. 最终输出：$\hat{\mathbf{y}} = \mathbf{a}^{[L]}$
4. 计算损失：$\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$

计算图

PyTorch自动构建动态计算图：

```
x = torch.randn(32, 784)  # 32个MNIST图像批次（展平）
model = TwoLayerNet(784, 256, 10)

# 前向传播构建计算图
logits = model(x)
predictions = F.softmax(logits, dim=1)
```

1.6 PyTorch简介

关键概念

概念	描述
张量	多维数组（类似NumPy）
nn.Module	所有神经网络模块的基类
Autograd	自动微分系统
优化器	基于梯度更新权重
DataLoader	批次处理和打乱数据

基本训练循环结构

```
for epoch in range(num_epochs):
    for batch_x, batch_y in dataloader:
        # 前向
        predictions = model(batch_x)
        loss = criterion(predictions, batch_y)

        # 反向（我们将在下一课详细介绍）
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

1.7 选择网络架构

模型选择决策框架

步骤1：识别您的数据类型

数据类型	架构家族	典型首选
表格（电子表格、数据库）	全连接MLP	2-3隐藏层，128-512神经元
图像	CNN	ResNet-18/34开始，ResNet-50用于精度
序列（文本、时间序列）	RNN/LSTM或Transformer	小数据用LSTM，大数据用Transformer
图（分子、社交网络）	GNN	GCN、GAT、GraphSage
多模态（图像+文本）	混合	视觉编码器 + 文本解码器

步骤2：确定模型大小指南

```
模型大小决策树：
========================

数据集大小？        期望精度？       计算预算？     架构
     │                      │                      │
     ├→ 小（~1K）      ├→ 快速原型  ├→ 有限       → 从基线开始
     │   样本           │   / 基线       │   （CPU/单   → 更小模型，
     │                      │                      │     GPU）      → 更多正则化
     │                      │                      │
     ├→ 中（~10K）    ├→ 好的结果    ├→ 中等      → 标准架构
     │   到 100K       │                      │   （单GPU）   → 中等深度
     │                      │                      │
     └→ 大（~1M+）     └→ 最先进      └→ 充足      → 大预训练
         样本                                     │   （多GPU）   → 微调或
                                                     │                    → 从头训练
```

模型大小的通用启发式：

1. **从小开始，然后扩展：**
    - 从简单的基线开始（例如逻辑回归、小MLP）
    - 在深入之前建立性能基线
    - 逐渐增加复杂度直到验证性能平台期
2. **过拟合规则：**
    - 如果训练损失 << 验证损失 → 模型太大/数据不够
    - 如果两者损失都高 → 模型太小/需要更多容量
    - 目标：两者应收敛到相似值
3. **参数数量指南（非常粗略）：**
    - 小问题（MNIST、简单分类）：10K - 100K参数
    - 中等问题（CIFAR-10、情感分析）：100K - 10M参数
    - 大问题（ImageNet、语言建模）：10M - 1B+参数
    - 注意：预训练模型通常需要更少的任务特定参数
4. **深度 vs 宽度权衡：**
    - **宽度**（每层更多神经元）：更擅长记忆，更容易优化
    - **深度**（更多层）：更好的层次特征学习，对复杂模式更有效
    - 现代趋势：更深（ResNet-152有152层）带跳跃连接

步骤3：架构经验法则

```
MLP:
- 输入层：匹配特征维度
- 隐藏层：从2层开始，宽度为输入大小的2-4倍
- 输出层：匹配类别数量（分类）或1（回归）
- 添加层直到验证性能停止改善

CNN:
- 简单数据集从3-5个卷积层开始
- 每次池化后通道数翻倍（例如 32 → 64 → 128 → 256）
- 几乎只使用3×3核（VGG经验）
- 在卷积后、激活前添加批归一化

Transformers:
- Base大小（BERT-base）：12层，768隐藏，12头
- Large大小（BERT-large）：24层，1024隐藏，16头
- 除非有100M+样本，否则不要从头训练
```

1.8 为什么GPU对深度学习很重要

计算现实

矩阵乘法是瓶颈：

```
一个适度网络中的简单前向传播：
- 输入：1000维向量
- 隐藏层：1000个神经元 → 1000×1000 = 100万次乘加
- 10个这样的层 → 1000万次操作
- 100个样本的批次 → 10亿次操作
- 100K样本上的一个训练epoch → 1万亿次操作
- 100个epoch → 100万亿次操作
```

CPU vs GPU架构：

```
CPU（Intel Xeon，32核）：          GPU（NVIDIA A100，6912核）：
┌─────────────────────┐              ┌────────────────────────────────┐
│  ████ 缓存          │              │  [█][█][█][█][█][█][█][█][█]  ← 数千    │
│  ████ 控制          │              │  [█][█][█][█][█][█][█][█][█]    个简单  │
│  ████ 逻辑          │              │  [█][█][█][█][█][█][█][█][█]    核心    │
│                      │              │   [█][█][█][█][█][█][█][█]             │
│  ████ 算术          │              │                                        │
│      （少，复杂）   │             │  每个核心小缓存，优化用于             │
│                      │              │   并行矩阵操作                        │
└─────────────────────┘              └────────────────────────────────┘

优化用于：                           优化用于：
- 顺序任务                          - 并行任务
- 分支预测                          - 相同操作处理多数据（SIMD）
- 低延迟                             - 高吞吐量
- 复杂控制流                        - 简单、重复计算
```

性能比较（典型）：

操作	CPU（1核）	CPU（16核）	GPU（A100）	加速比
矩阵乘法（1K×1K）	1秒	0.1秒	0.001秒	**1000×**
训练ResNet-50（1 epoch）	24小时	2小时	2分钟	**720×**
训练GPT-3（1步）	不可能	数周	数小时	**使其成为可能**

GPU在神经网络中表现出色的原因：

1. **大规模并行：**
    - 矩阵乘法可以分解为数千个独立的点积
    - GPU有1000+核心 vs CPU的10+核心
    - 与神经网络操作完美匹配
2. **内存带宽：**
    - GPU有1-2 TB/s内存带宽 vs CPU的~100 GB/s
    - 神经网络通常是内存受限，而非计算受限
3. **专用硬件：**
    - **Tensor Cores**（Volta及以后）：在一个周期内执行4×4矩阵乘法
    - **混合精度支持：** FP16/BF16训练，FP32累加
    - **NVLink/Infinity Fabric：** 分布式训练的快速GPU间通信

什么时候可以使用CPU：

```
✓ 小模型上的模型推理（MobileNet、DistilBERT）
✓ 训练非常小的网络（< 100K参数）
✓ 数据预处理和特征工程
✓ 用玩具示例调试和原型设计

× 训练任何生产深度学习模型
× 训练大于64×64图像的CNN
× 训练具有> 1M参数的Transformers
```

云GPU选项（截至2025年）：

提供商	入门级	中端	高端
Google Colab	免费T4	Pro: V100	Pro+: A100
AWS	g4dn.xlarge (T4)	p3.2xlarge (V100)	p4d.24xlarge (8×A100)
Lambda Labs	1× RTX 3090	4× A6000	8× A100
大学	共享集群	实验室服务器	HPC中心

实用建议：

- 学习从Colab（免费）或大学资源开始
- 严肃研究使用云积分或大学HPC集群
- 生产环境考虑云成本 vs 购买硬件

- 总是使用混合精度（FP16/BF16）（如果可用）→ 2倍速度，2倍内存

1.9 实验：你的第一个神经网络

A部分：NumPy实现（30分钟）

仅使用NumPy从头开始实现一个2层网络：

1. 初始化权重和偏置
2. 实现前向传播
3. 计算交叉熵损失
4. （预览）手动计算梯度

起始代码：

```
import numpy as np

class NumPyNN:
    def __init__(self, input_size, hidden_size, output_size):
        self.W1 = np.random.randn(input_size, hidden_size) * 0.01
        self.b1 = np.zeros(hidden_size)
        self.W2 = np.random.randn(hidden_size, output_size) * 0.01
        self.b2 = np.zeros(output_size)

    def forward(self, X):
        # 在此实现
        pass

    def compute_loss(self, y_pred, y_true):
        # 交叉熵损失
        pass
```

B部分：PyTorch实现

使用PyTorch实现相同的网络：

1. 定义`nn.Module`类
2. 使用内置损失函数
3. 在MNIST或合成数据上训练
4. 可视化预测

练习：

1. 尝试不同的隐藏层大小
2. 尝试不同的激活函数
3. 绘制训练损失曲线
4. 可视化权重矩阵

关键要点

1. 神经网络将简单的计算单元（神经元）组合成强大的函数近似器
2. ReLU是隐藏层的默认激活函数；softmax/sigmoid用于输出
3. 损失函数定义"好"预测的含义
4. PyTorch等框架自动处理计算图
5. 在学习反向传播之前，理解前向传播的数学原理至关重要

延伸阅读

书籍和课程

1. Neural Networks and Deep Learning - Michael Nielsen（免费在线书籍）
2. PyTorch 60-Minute Blitz
3. CS231n Lecture 4 - 反向传播（下一课预览）

可视化学习资源

4. **Andrej Karpathy的Neural Networks: Zero to Hero** - 逐步视频系列从头开始构建神经网络。这些笔记中的计算图可视化受Karpathy的*micrograd*方法启发。
5. **Micrograd** - 一个微小的autograd引擎，在动态构建的DAG上实现反向传播。强烈推荐用于理解自动微分的底层工作原理。

可视化工具

6. **Netflow.js** - 交互式神经网络可视化（参考HMKCODE的反向传播教程）。用于逐步可视化前向和后向传播。

作业1预览

- 在NumPy中实现3层MLP
- 在简单数据集上训练
- 尝试不同架构
- 撰写分析报告分析结果